EXPERIMENT NO 3

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
// Structure to represent an item
struct Item {
    double value;
    double weight;
};
// Comparator function to sort items by value/weight ratio
bool cmp(Item a, Item b) {
    double r1 = a.value / a.weight;
    double r2 = b.value / b.weight;
    return r1 > r2; // Descending order
}
// Function to solve fractional knapsack
double fractionalKnapsack(int n, double W, vector<Item> &items) {
    // Sort items by value/weight ratio
    sort(items.begin(), items.end(), cmp);
double totalValue = 0.0;
   for (int i = 0; i < n; i++) {
        if (items[i].weight <= W) {
            // Take whole item
            W -= items[i].weight;
            totalValue += items[i].value;
        } else {
            // Take fractional part
            totalValue += items[i].value * (W / items[i].weight);
            break; // Knapsack is full
```

```
    }
  }
```

OUTPUT

```
icoer@icoer-Vostro-3470:~$ g++ fractional_knapsack.cpp
icoer@icoer-Vostro-3470:~$ ./a.out
Enter number of items: 3
Enter value and weight of each item:
60
10
100
20
120
30
Enter capacity of knapsack: 50
Maximum value in the knapsack = 240
```